

```

    ❷ //printMaze(maze); // Uncomment to view each backtrack step.
    return false; // BASE CASE
}

printMaze(MAZE);
solveMaze(MAZE);
printMaze(MAZE);
</script>

```

A lot of this code is not directly related to the recursive maze-solving algorithm. The `MAZE` variable stores the maze data as a multiline string with hashtags to represent walls, an `s` for the starting point, and an `e` for the exit. This string is converted into a list that contains lists of strings, with each string representing a single character in the maze. This allows us to access `MAZE[y][x]` (note that `y` comes first) to get the character at the `x, y` coordinate in the original `MAZE` string. The `printMaze()` function can accept this list-of-list data structure and display the maze on the screen. The `findStart()` function accepts this data structure and returns the `x, y` coordinates of the `s` starting point. Feel free to edit the maze string yourself—although remember that, in order for the solving algorithm to work, the maze cannot have any loops.

The recursive algorithm is in the `solveMaze()` function. The arguments to this function are the maze data structure, the current `x-` and `y-`coordinates, and a `visited` list (which is created if none was supplied) ❶. The `visited` list contains all the coordinates that have previously been visited so that when the algorithm backtracks from a dead end to an earlier intersection, it knows which paths it has tried before and can try a different one. The path from the start to the exit is marked by replacing the spaces (matching the `EMPTY` constant) in the maze data structure with periods (from the `PATH` constant).

The maze-solving algorithm is similar to our flood fill program in Chapter 3 in that it “spreads” to neighboring coordinates, though when it reaches a dead end, it backtracks to an earlier intersection. The `solveMaze()` function receives the `x, y` coordinates indicating the algorithm’s current location in the maze. If this is the exit, the function returns `True`, causing all the recursive calls to also return `True`. The maze data structure remains marked with the solution path.

Otherwise, the algorithm marks the current `x, y` coordinates in the maze data structure with a period and adds the coordinates to the `visited` list ❷.